Locking, Timestamp-based schedulers, Optimistic Concurrency Control

Concurrency Control Protocols

- The concurrency control protocols ensure the *atomicity, consistency, isolation, durability* and *serializability* of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:

- Lock Based Concurrency Control Protocol

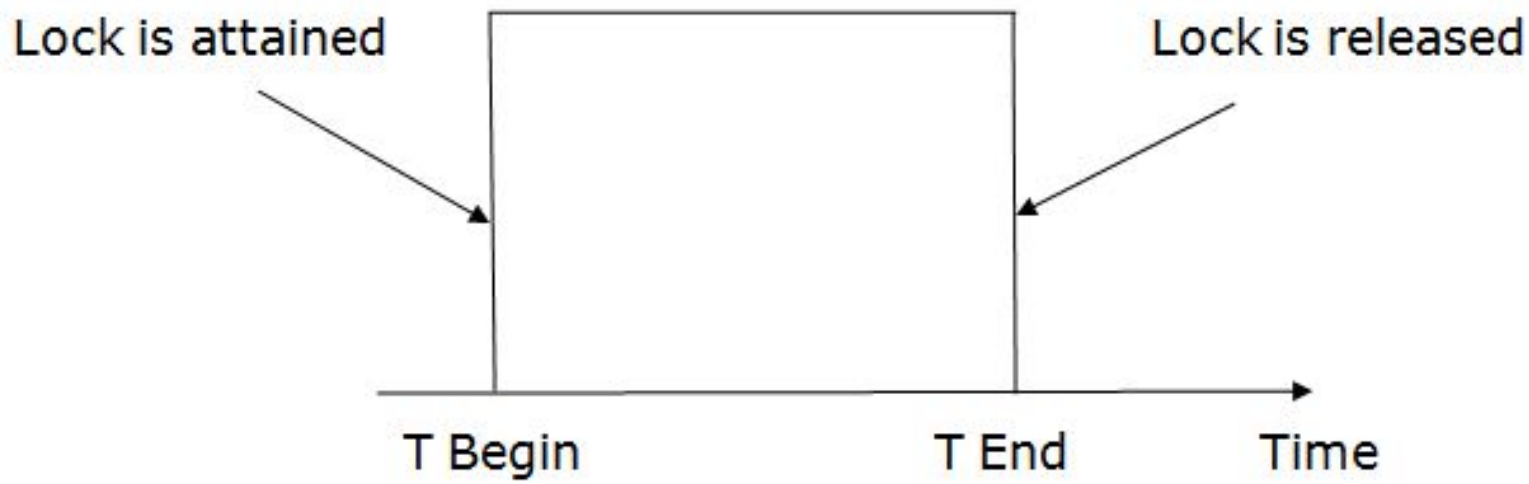
- Time Stamp Concurrency Control Protocol

- Validation Based Concurrency Control Protocol

Lock-Based Protocol

- In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:
- **1. Shared lock:**
- It is also known as a Read-only lock. In a shared lock, the data item can only read by the transaction.
- It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.
- **2. Exclusive lock:**
- In the exclusive lock, the data item can be both reads as well as written by the transaction.
- This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously.

Lock-Based Protocol



Timestamp Ordering Protocol

The **Timestamp Ordering Protocol** in a **Database Management System (DBMS)** is a concurrency control technique that uses timestamps to order transactions and ensure serializability (i.e., consistency without conflicts) in a multi-user environment. This protocol helps manage simultaneous transaction access to the same data without conflicts or anomalies.

Key Concepts of Timestamp Ordering Protocol

1.Timestamps:

- Each transaction is assigned a unique **timestamp** when it starts. This timestamp reflects the transaction's start time, helping to order transactions chronologically.
- **Oldest transactions** get **earlier timestamps**, while **newer transactions** get **later timestamps**.

2.Read and Write Timestamps:

- Each data item has two timestamps associated with it:
  - **Read Timestamp (RTS)**: The timestamp of the last transaction that successfully read the data item.
  - **Write Timestamp (WTS)**: The timestamp of the last transaction that successfully wrote to the data item.

Timestamp Ordering Protocol

3.Rules of Timestamp Ordering:

- **For Read Operations** (T wants to read data item X):
  - If TS(T) < WTS(X): This means T is too old (a later transaction has already modified X), so T must be **aborted** to avoid reading stale data.
  - If TS(T) ≥ WTS(X): The read is **allowed**, and RTS(X) is updated to TS(T).

- **For Write Operations** (T wants to write data item X):
  - If TS(T) < RTS(X): This means T is too old (a later transaction has already read X), so T must be **aborted** to prevent overwriting newer data.
  - If TS(T) < WTS(X): This means a later transaction has already written to X, so T is **aborted**.
  - If TS(T) ≥ WTS(X) and TS(T) ≥ RTS(X): The write is **allowed**, and WTS(X) is updated to TS(T).

Timestamp Ordering Protocol

4.Abort and Restart Mechanism:

- When a transaction is **aborted** due to a timestamp conflict, it is typically restarted with a **new, larger timestamp** to allow it to execute after the conflicting transactions.

Advantages of Timestamp Ordering Protocol

- **Ensures Serializability**: Transactions appear to execute in a serial order based on their timestamps, preventing inconsistencies.
- **No Deadlocks**: Since it uses a non-blocking approach (transactions are aborted and restarted rather than waiting), it avoids deadlock situations.

Disadvantages of Timestamp Ordering Protocol

- **Frequent Aborts**: Transactions are aborted and restarted if conflicts arise, which can lead to high overhead and wasted work.
- **Older Transactions Disadvantage**: Transactions with older timestamps are more likely to be aborted if they conflict with newer transactions.

Validation Based Protocol

The **Validation-Based Protocol**, also known as the **Optimistic Concurrency Control Protocol**, is a concurrency control method used in **Database Management Systems (DBMS)**. This protocol assumes that conflicts among transactions are rare, so it lets transactions execute without strict locks or checks at the start and only validates them at the end to see if they can be committed safely. This approach is often used when the likelihood of conflict is low, providing a more efficient way to handle concurrent transactions.

Phases of Validation-Based Protocol

The protocol divides transaction execution into three main phases:

1. **Read Phase**:
2. **Validation Phase** (or **Certification Phase**):
3. **Write Phase**:

Validation Based Protocol

Phases of Validation-Based Protocol
The protocol divides transaction execution into three main phases:

1. **Read Phase**:
    1. In this phase, the transaction reads data items from the database and **performs operations** in its local workspace without making any changes to the actual database.
    2. All updates are made in a temporary space (buffer or local memory), isolating them from the main database.

2. **Validation Phase** (or **Certification Phase**):
    1. Once the transaction completes its read and write operations, it enters the validation phase.
    2. The DBMS checks whether committing this transaction would conflict with other concurrently executed transactions. This is done by examining if the transaction's read and write operations interfere with those of other transactions.
    3. If the transaction passes the validation check, it can proceed to the write phase. If it fails, it is aborted and may restart.

3. **Write Phase**:
    1. If the transaction successfully passes validation, it enters the write phase.
    2. The transaction's changes are **applied to the database**, making the updates permanent. If it fails validation, the transaction is aborted, and none of its changes are applied.

Validation Based Protocol

Validation Conditions
To ensure serializability (consistent results as if transactions were executed sequentially), the validation-based protocol uses timestamps and checks based on three primary conditions. These conditions vary depending on whether transactions follow:

- **Backward Validation**:
  - Check if the current transaction conflicts with any previously committed transactions.
- **Forward Validation**:
  - Check if the current transaction conflicts with transactions that are yet to be validated.
- **Mixed Validation**:
  - Checks are a combination of backward and forward validation

The most common approach uses **Backward Validation**, where a transaction T_i passes validation if one of the following holds for each committed transaction T_j with $TS(T_j) < TS(T_i)$:

- T_j's write phase finished before T_i started, or
- T_i's read phase finished before T_j started.